

AI FitTrack Requirement Analysis

Phase 2

This document converts the official requirements into verifiable backend requirements and maps each one to the uploaded implementation and supplied evidence.

Field	Value
Project	AI FitTrack
Submission	SmartBridge Naan Mudhalvan
Team ID	SWTID-2026-8100
Team size	4
Team Leader	Parthiban M
Team Members	Naveen J; Nidesh N S; Nithish Kumar P
Project scope	Backend-only REST API
Technology	Node.js, Express.js, MongoDB, Mongoose, JWT and Google Gemini AI
Source baseline	Latest uploaded backend ZIP and supplied verification evidence
Document status	Submission-ready technical content; individual academic IDs and final links remain pending

Prepared for academic evaluation. Secrets, credentials, private environment files and unverified claims are excluded.

Document contents

1. Requirement sources and scope
2. Actors and workflows
3. User stories
4. Functional requirements
5. Nonfunctional requirements
6. Technology stack
7. Data rules
8. Traceability and open items

Requirement sources and scope

The official AI FitTrack document and the uploaded backend agree on the required backend domain: secure user authentication, user-owned workout management, workout search, MongoDB persistence and Google Gemini AI guidance. The official document also mentions possible frontend consumers, but this submission is intentionally backend-only.

Functional requirement groups

All protected features are scoped to the authenticated user

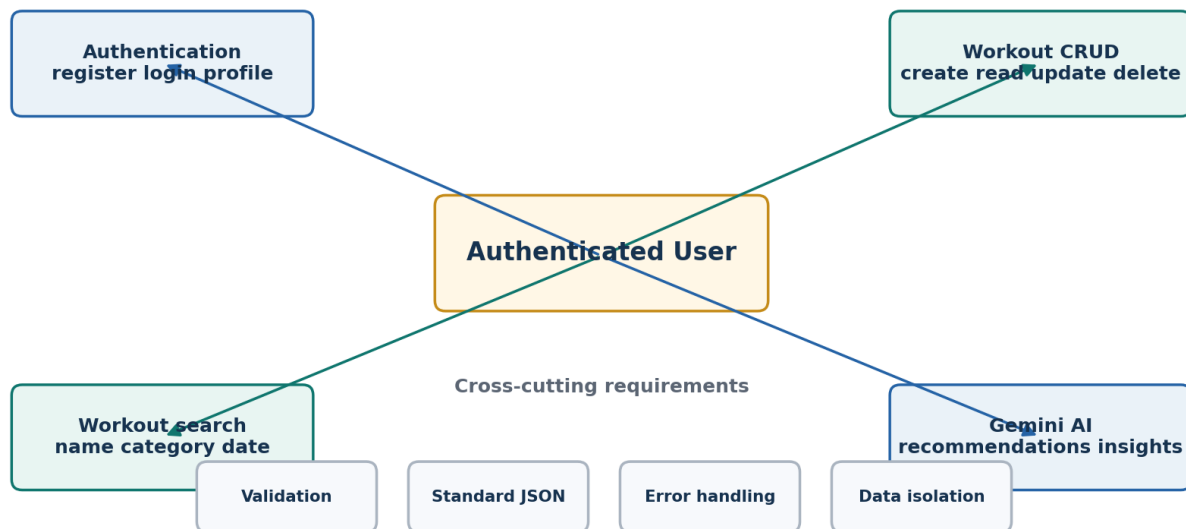


Figure 1 Functional requirement groups and cross-cutting controls

Primary actor

The only implemented role is an authenticated user. Public operations are registration, login and email-availability checking. Profile, workout and AI operations require a valid JWT. No administrator role or team-management feature exists in the uploaded source.

End-to-end user workflow

1. Register with a valid name, email and password.
2. Log in and receive a Bearer JWT.
3. Use the token to retrieve the profile and access protected routes.
4. Create, list, retrieve, update, search and delete owned workout records.
5. Request a Gemini workout plan using age, fitness goal and experience level.
6. Request fitness insights using submitted statistics or owner-scoped MongoDB statistics.

User stories and acceptance criteria

ID	User story	Acceptance criterion	Priority
US-01	As a visitor, I can register and log in securely.	Valid details return a safe user and JWT; duplicate or invalid details are rejected.	High
US-02	As an authenticated user, I can view my profile.	A valid Bearer token returns only the current safe user fields.	High
US-03	As an authenticated user, I can manage my workout records.	Create, list, retrieve, update and delete operate only on the token owner's records.	High
US-04	As an authenticated user, I can search my history.	Name, category and valid UTC-date filters return only matching owned workouts.	High
US-05	As an authenticated user, I can request a workout recommendation.	Validated age, goal and experience produce a structured general-fitness response.	High
US-06	As an authenticated user, I can request fitness insights.	Submitted or database-derived statistics produce structured insights without exposing another user.	High

Functional requirement traceability

ID	Requirement	Verified status
R1	Layered MVC-style backend with routes, controllers, models, middleware, services and config	Implemented

ID	Requirement	Verified status
R2	Node.js and Express.js REST API	Implemented
R3	MongoDB and Mongoose connection management	Implemented
R4	Users and workouts collections	Implemented
R5	User schema with validation, unique email, bcrypt and timestamps	Implemented
R6	Workout schema with owner, fields, validation and timestamps	Implemented
R7	User registration and duplicate handling	Implemented and live-evidenced
R8	User login and JWT issuance	Implemented and live-evidenced
R9	Protected user profile	Implemented and live-evidenced
R10	Create owner-scoped workout	Implemented and live-evidenced
R11	List workout history	Implemented and live-evidenced
R12	Search by name, category and date	Implemented and live-evidenced
R13	Get workout by identifier	Implemented and live-evidenced
R14	Partial workout update	Implemented and live-evidenced
R15	Delete workout and deny later retrieval	Implemented and live-evidenced
R16	Gemini workout recommendation	Implemented and live-evidenced
R17	Gemini fitness insights using submitted or database statistics	Implemented and live-evidenced
R18	Standard JSON responses and centralized error handling	Implemented
R19	Request and schema validation	Implemented and automated-test evidenced
R20	JWT middleware and fresh user lookup	Implemented and automated-test evidenced

ID	Requirement	Verified status
R21	Password protection and response redaction	Implemented and live-evidenced
R22	Per-user data isolation	Implemented and automated-test evidenced
R23	Postman, automated, smoke and Atlas verification	Evidence supplied
R24	Environment-based MongoDB, JWT and Gemini configuration	Implemented with example only

Nonfunctional requirements

Category	Requirement	Implementation evidence
Security	Hash passwords, protect routes, isolate records and keep secrets outside source control.	bcrypt cost 12, JWT middleware, owner-scoped queries, .gitignore and .env.example.
Reliability	Fail clearly when database, token or AI configuration is unavailable.	Database-gated startup, fail-fast JWT secret check, centralized error middleware and normalized Gemini errors.
Maintainability	Separate routing, business logic, data models and external services.	routes, controllers, models, middleware, services, config and utils folders.
Performance	Bound list results and avoid uncontrolled external retry loops.	Workout limit clamped to 1-500; Gemini retry count and delay are bounded.
Interoperability	Use HTTP and JSON for frontend or mobile integration.	Express REST endpoints, CORS and standardized response envelope.
Testability	Support isolated automated testing and live API verification.	Node test suites, Supertest, MongoDB memory server, smoke scripts and Postman collection.

Technology stack

Component	Implemented technology	Purpose
Runtime and API	Node.js 20+ and Express.js 5	REST routing, middleware and server lifecycle
Database	MongoDB with Mongoose 9	Validated users and workout persistence
Authentication	JWT and bcryptjs	Signed access tokens and password hashing
Artificial intelligence	Google Gemini through @google/genai	Structured workout recommendations and fitness insights
Verification	Node test runner, Supertest, MongoDB Memory Server and Postman	Automated, isolated and guided API testing

Software and execution requirements

Item	Required baseline
Runtime	Node.js 20 or later, because the uploaded package.json declares node >=20.0.0.
Package manager	npm with dependencies installed from server/package-lock.json.
Database	MongoDB Atlas or a compatible local MongoDB deployment.
API client	Postman, curl or another HTTP client.
AI provider	Google Gemini API access configured through GEMINI_API_KEY.
Development editor	Visual Studio Code or equivalent.

Data validation rules

Object	Field	Implemented rule
User	name	Required; trimmed; 2 to 80 characters.
User	email	Required; normalized lowercase; regular-expression validation; unique index.
User	password	Required; minimum 6 characters; bcrypt hash; excluded from normal queries and serialized output.
Workout	workoutName	Required; trimmed; 2 to 120 characters.
Workout	category	One of Cardio, Strength Training, Yoga, Running, Cycling or Walking.
Workout	duration	Number from 1 to 1440 minutes.
Workout	caloriesBurned	Number from 0 to 20000.
Workout	workoutDate	Required valid date.
AI recommendation	age	Whole number from 13 to 100.
AI recommendation	goal and experience	Case-insensitive values from implemented allow-lists.
AI insights	statistics	Bounded numeric metrics, or useDatabaseStats true.

Requirement interpretation notes

- The official software list contains a stale sentence about posts, categories and comments. The same document's schema tables and the source correctly use users and workouts.
- The official document shows Node.js 16 or above, while the delivered package explicitly requires Node.js 20 or above. The implementation requirement takes precedence for execution.
- The implementation accepts compatibility aliases for experience and averageDuration while retaining canonical API names.
- Search is supported on both GET /api/workouts and GET /api/workouts/search.

- The public email-availability endpoint is an implemented extension beyond the core official endpoint list.

Items requiring student confirmation

Pending item	What to provide
Individual identifiers	The four names and team designations are included. NM IDs and register or roll numbers were not supplied.
Institution details	College name, academic year, guide or mentor name and official submission date need confirmation.
Responsibility allocation	Team Leader and Team Member designations are recorded, but module ownership and task responsibilities were not supplied.
Repository	Insert the final public GitHub repository URL after upload.
Deployment	Insert the deployed API base URL if the evaluator requires a hosted demonstration.